

LAPORAN

Analisis Menggunakan Data Mining Market Basket Streamlit Cafe KM02 Coffee And Working Space



Disusun oleh :

NAMA : REZA DWI PUSPITA

NIM : 21523076

UNIVERSITAS ISLAM INDONESIA

YOGYAKARTA

2024

DAFTAR ISI

BAB 1 PENDAHULUAN.....	1
1.1 Deskripsi.....	1
1.2 Fitur.....	1
BAB II PEMBAHASAN.....	3
2.1 Data Mining.....	3
2.1.1 Penjelasan Data Mining.....	3
2.1.2 Interpretasi Hasil.....	3
2.1.3 Skenario Pemanfaatan dan Penggunaan Visualisasi.....	3
2.2 Proses Market Basket Analysis.....	5
2.2.1 Membuat dan Memeriksa Data Awal.....	5
2.2.2 Pra-Pemrosesan Data.....	6
2.2.3 Visualisasi Data.....	8
2.2.4 Aturan Asosiasi.....	10
2.2.5 Penerapan Algoritma Apriori.....	13
2.3 Mengintegrasikan Market Basket Analysis ke dalam Streamlit.....	15
BAB III PENUTUP.....	17
3.1 Kesimpulan.....	17

BAB 1 PENDAHULUAN

1.1 Deskripsi

Dataset yang dipilih untuk tugas ini adalah dataset yang bernama “KM02 Coffee And Working Space” yang saya ambil dari data internal cafe yang berada di Semarang. Dataset ini mencatat transaksi penjualan di kafe "KM02 Coffee And Working Space" selama 1 bulan di bulan Juni tahun 2023. Dataset ini berisi data penjualan cafe, yang mencakup informasi mengenai produk yang dibeli, waktu pembelian, dan pembayaran, dll.

Tujuan dari dataset penjualan di kafe "KM02 Coffee And Working Space" adalah untuk menyediakan gambaran tentang aktivitas penjualan harian yang dapat digunakan untuk analisis bisnis dan pengambilan keputusan strategis. Dalam konteks bisnis dataset untuk memberikan wawasan yang berharga tentang kinerja penjualan kafe dan preferensi pelanggan terhadap produk makanan dan minuman. Dengan pemahaman ini, kafe dapat mengoptimalkan pendapatan penjualan dan keuntungan dengan menyediakan produk yang paling diminati oleh pelanggan dari berbagai kategori seperti minuman kopi, teh, makanan ringan, dll. Selain itu, dataset ini juga dapat membantu dalam manajemen persediaan dengan memperhitungkan permintaan produk tertentu yang tinggi, sehingga kafe dapat memastikan stok barang tersedia secara optimal untuk memenuhi kebutuhan pelanggan. Dengan menganalisis data tersebut, cafe dapat membuat keputusan strategis untuk memberikan diskon kepada pelanggan yang dapat meningkatkan keuntungan.

1.2 Fitur

Dataset bernama “KM02 Coffee And Working Space” memiliki jumlah baris sebanyak 2762 dan kolom yang berjumlah 25. Dataset ini memiliki ukuran 440KB dalam bentuk csv. Berikut fitur-fitur dari dataset tersebut:

- a. Outlet: Nama outlet cafe.
- b. Receipt Number: Nomor tanda terima transaksi.
- c. Date: Tanggal transaksi.
- d. Time: Waktu transaksi.

- e. Category: Kategori item yang dijual (misalnya: drink coffee, drink tea, dll).
- f. Brand: Merek dari item yang dijual.
- g. Items: Nama item yang dijual.
- h. Variant: Varian dari item yang dijual.
- i. SKU: Unit kode stok.
- j. Quantity: Jumlah item yang terjual.
- k. Modifier Applied: Modifikasi yang diterapkan pada item.
- l. Discount Applied: Diskon yang diterapkan pada item.
- m. Gross Sales: Penjualan kotor sebelum diskon dan pajak.
- n. Discounts: Jumlah diskon yang diberikan.
- o. Refunds: Jumlah pengembalian uang.
- p. Net Sales: Penjualan bersih setelah diskon.
- q. Gratuity: Tip yang diberikan.
- r. Tax: Pajak yang dikenakan pada penjualan.
- s. Sales Type: Tipe penjualan (misalnya:dine in, take away, grab food).
- t. Collected By: Nama kasir yang menangani transaksi.
- u. Served By: Nama staf yang melayani.
- v. Customer: Informasi tentang pelanggan.
- w. Payment Method: Metode pembayaran (misalnya: tunai, Gopay, Mandiri, dll).
- x. Event Type: Jenis acara.
- y. Reason of Refund: Alasan pengembalian uang.

BAB II PEMBAHASAN

2.1 Data Mining

2.1.1 Penjelasan Data Mining

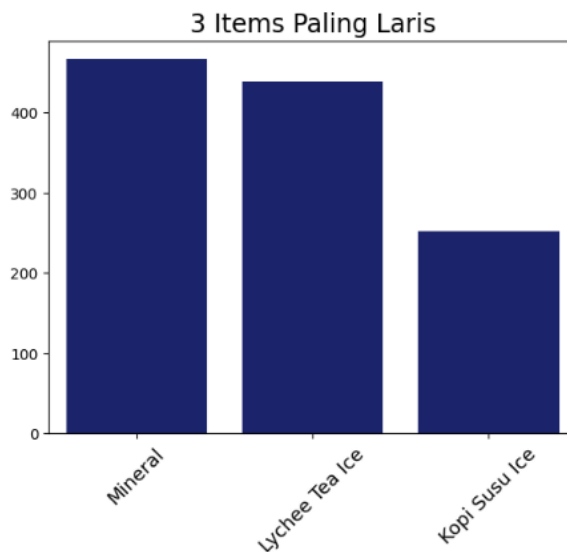
Algoritma Apriori adalah teknik data mining yang digunakan untuk menemukan itemsets yang sering muncul dalam data transaksi dan untuk menghasilkan aturan asosiasi. Algoritma ini cocok digunakan dalam analisis keranjang belanja untuk mengidentifikasi produk yang sering dibeli bersamaan. Teknik ini dipilih karena dapat membantu memahami pola pembelian pelanggan dan memberikan wawasan yang berguna untuk strategi pemasaran dan penjualan.

2.1.2 Interpretasi Hasil

Hasil dari penerapan algoritma Apriori memberikan daftar itemsets yang sering muncul bersama dalam data penjualan. Aturan-aturan asosiasi yang dihasilkan dapat digunakan untuk membuat rekomendasi produk, merancang promosi yang lebih efektif, dan mengoptimalkan tata letak toko.

2.1.3 Skenario Pemanfaatan dan Penggunaan Visualisasi

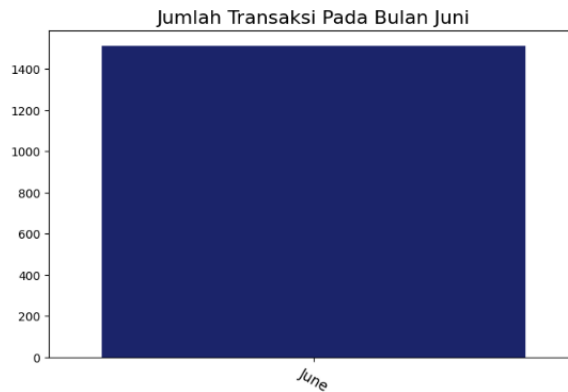
a. 3 Item Terlaris



Visualisasi ini membantu manajemen cafe untuk mengetahui produk mana yang paling populer di kalangan pelanggan. Informasi ini dapat digunakan

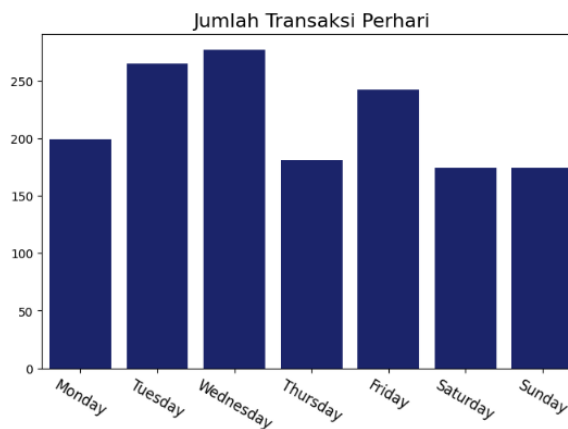
untuk mengelola persediaan dan merancang promosi yang menargetkan item-item populer tersebut.

b. Jumlah Transaksi Pada Bulan Juni



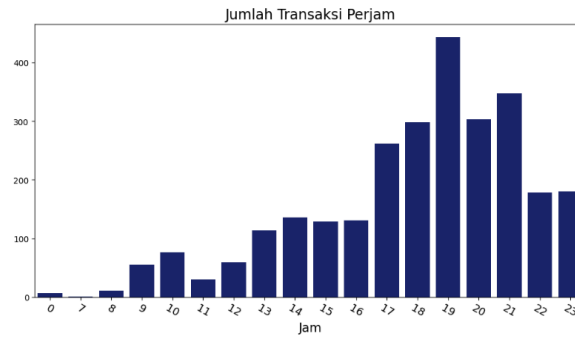
Visualisasi ini memberikan gambaran tentang kinerja penjualan selama bulan tertentu, dalam hal ini bulan Juni. Informasi ini berguna untuk memahami pola musiman dan merencanakan strategi penjualan untuk bulan-bulan mendatang.

c. Jumlah Transaksi Per Hari:



Visualisasi ini membantu memahami hari-hari mana dalam seminggu yang memiliki volume penjualan tertinggi. Informasi ini dapat digunakan untuk mengatur jadwal karyawan dan memastikan stok yang cukup pada hari-hari sibuk.

d. Jumlah Transaksi Per Jam



Visualisasi ini memberikan wawasan tentang jam-jam sibuk dalam sehari. Informasi ini dapat digunakan untuk mengatur shift karyawan dan memastikan layanan yang efisien selama jam-jam puncak.

e. Pola Pembelian Produk di Kafe Menggunakan Algoritma Apriori

	antecedents	consequents	support	confidence	lift
1072	(americano hot)	(indomie rebus km02, lemon tea hot, americano ...	0.054054	1.0	18.5
874	(lemon tea ice, lychee tea ice)	(indomie rebus km02, jamur crispy)	0.054054	1.0	18.5
1519	(americano ice)	(lemon tea ice, lemon tea hot, jamur crispy, L...	0.054054	1.0	18.5
891	(indomie rebus km02, lychee tea ice)	(lemon tea ice, lemon tea hot)	0.054054	1.0	18.5
890	(lemon tea hot, lychee tea ice)	(lemon tea ice, indomie rebus km02)	0.054054	1.0	18.5
1525	(custom amount, jamur crispy, kopi susu ice, L...	(tempe)	0.054054	1.0	18.5
1529	(custom amount, kopi susu ice, lychee tea ice)	(tempe, jamur crispy)	0.054054	1.0	18.5
887	(lemon tea ice, indomie rebus km02)	(lemon tea hot, lychee tea ice)	0.054054	1.0	18.5
886	(lemon tea ice, lemon tea hot)	(indomie rebus km02, lychee tea ice)	0.054054	1.0	18.5
1530	(tempe, kopi susu ice, lychee tea ice)	(custom amount, jamur crispy)	0.054054	1.0	18.5

Setiap baris dalam tabel visualisasi ini menunjukkan hubungan antara satu set item yang dibeli bersama dalam transaksi (antecedents) dan item-item yang biasanya dibeli bersamaan dengan set tersebut (consequents).

Informasi ini sangat berguna untuk berbagai tujuan bisnis. Pertama, manajemen kafe dapat menggunakan pola-pola ini untuk membuat bundel produk yang menarik, menawarkan diskon untuk kombinasi produk tertentu yang sering dibeli bersama, atau merancang promosi spesial. Misalnya, jika "americano hot" sering dibeli bersama dengan "indomie rebus km02, lemon tea hot, dan americano ice", maka kafe dapat menawarkan paket diskon untuk pembelian kombinasi tersebut.

Kedua, visualisasi ini membantu dalam pengaturan tata letak produk di dalam kafe. Produk-produk yang sering dibeli bersama dapat ditempatkan berdekatan untuk memudahkan pelanggan menemukan dan membeli kombinasi produk tersebut. Ini dapat meningkatkan efisiensi pembelian pelanggan dan berpotensi meningkatkan penjualan.

Ketiga, pemahaman mendalam tentang pola pembelian ini juga membantu dalam pengelolaan inventaris. Manajemen kafe dapat lebih akurat memprediksi permintaan dan mengelola stok produk dengan lebih efektif, menghindari kekurangan atau kelebihan stok untuk produk-produk tertentu.

2.2 Proses Market Basket Analysis

2.2.1 Membuat dan Memeriksa Data Awal

Mulai dengan memuat dataset dan memeriksa bentuk serta beberapa baris pertama untuk memahami struktur dan isinya. Untuk ini, kita menggunakan pustaka seperti numpy, pandas, matplotlib, dan seaborn. Pertama-tama, memuat dataset menggunakan fungsi `pd.read_csv()` dan kemudian memeriksa bentuk data dengan `data.shape` yang memberikan informasi tentang jumlah baris dan kolom dalam dataset. Selanjutnya, kita menampilkan beberapa baris pertama dari dataset menggunakan `data.head()` untuk mendapatkan gambaran awal tentang isinya.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data = pd.read_csv('penjualancafe.csv')
print('DataFrame shape :', data.shape)
data.head()
```

Penjelasan:

- **Import Library:** Kita memulai dengan memuat pustaka yang diperlukan seperti numpy, pandas, matplotlib, dan seaborn.
- **Membaca Data:** Kita menggunakan `pd.read_csv()` untuk membaca dataset CSV yang berisi data penjualan kafe.

- **Memeriksa Bentuk Data:** `data.shape` memberikan informasi tentang jumlah baris dan kolom dalam dataset, yang membantu kita memahami ukuran dataset.
- **Menampilkan Baris Pertama:** `data.head()` menampilkan beberapa baris pertama dari dataset, yang memungkinkan kita untuk memeriksa dan memahami struktur serta isi data.

2.2.2 Pra-Pemrosesan Data

a. Konversi Tanggal dan Waktu:

Untuk bekerja secara efektif dengan data tanggal dan waktu, kita mengonversi kolom Date dan Time menjadi objek `datetime`. Ini penting agar kita bisa melakukan operasi dan analisis berbasis waktu dengan lebih mudah dan akurat. Pertama, kita menampilkan beberapa baris pertama dari kolom Date dan Time untuk memeriksa formatnya menggunakan `print(data[['Date', 'Time']].head())`. Setelah memastikan formatnya, kita mengonversi kolom Date dengan menggunakan `pd.to_datetime(data['Date'], format='%d-%m-%Y', errors='coerce')` dan kolom Time dengan `pd.to_datetime(data['Time'], format='%H:%M:%S', errors='coerce')`. Langkah ini memastikan bahwa data tanggal dan waktu dikonversi ke format yang tepat, sehingga memudahkan analisis lebih lanjut.

```
# Display the first few rows of the 'Date' and 'Time' columns to inspect their format
print(data[['Date', 'Time']].head())
```

	Date	Time
0	30-06-2023	22:31:21
1	30-06-2023	22:31:21
2	30-06-2023	22:31:21
3	30-06-2023	22:31:21
4	30-06-2023	22:31:21

```
#format data waktu
data['Date'] = pd.to_datetime(data['Date'], format='%d-%m-%Y', errors='coerce')
data['Time'] = pd.to_datetime(data['Time'], format='%H:%M:%S', errors='coerce')

# Check the result
print(data[['Date', 'Time']].head())
```

	Date	Time
0	2023-06-30 1900-01-01	22:31:21
1	2023-06-30 1900-01-01	22:31:21
2	2023-06-30 1900-01-01	22:31:21
3	2023-06-30 1900-01-01	22:31:21
4	2023-06-30 1900-01-01	22:31:21

Penjelasan:

- **Memeriksa Format Tanggal dan Waktu:** Langkah pertama adalah memeriksa format kolom Date dan Time dengan menampilkan beberapa

baris pertama. Ini membantu kita memahami bagaimana data tanggal dan waktu diatur dalam dataset.

- **Mengonversi Tanggal:** Fungsi `pd.to_datetime(data['Date'], format='%d-%m-%Y', errors='coerce')` digunakan untuk mengkonversi kolom Date ke dalam objek datetime dengan format yang benar (%d-%m-%Y). Parameter `errors='coerce'` memastikan bahwa setiap nilai yang tidak sesuai dengan format akan dikonversi menjadi NaT (Not a Time).
- **Mengonversi Waktu:** Demikian pula, kita menggunakan `pd.to_datetime(data['Time'], format='%H:%M:%S', errors='coerce')` untuk mengonversi kolom Time ke dalam objek datetime dengan format yang benar (%H:%M:%S).

b. Ekstraksi Komponen Tanggal dan Waktu:

Setelah kolom Date dan Time dikonversi ke dalam format datetime, kita dapat mengekstrak komponen-komponen seperti bulan, hari, dan jam dari kolom-kolom ini. Ekstraksi ini penting untuk analisis lebih lanjut, seperti mengidentifikasi tren penjualan berdasarkan bulan, hari dalam seminggu, dan jam dalam sehari.

```
data['month'] = data['Date'].dt.month
data['day'] = data['Date'].dt.weekday
data['hour'] = data['Time'].dt.hour

print(data.head())
```

Penjelasan:

- Ekstraksi Bulan: Kita mengekstrak bulan dari kolom Date menggunakan `data['Date'].dt.month`. Ini menambahkan kolom baru bernama month yang berisi informasi bulan dari setiap tanggal.
- Ekstraksi Hari: Kita mengekstrak hari dalam format numerik (0 untuk Senin, 6 untuk Minggu) dari kolom Date menggunakan `data['Date'].dt.weekday`. Ini menambahkan kolom baru bernama day.
- Ekstraksi Jam: Kita mengekstrak jam dari kolom Time menggunakan `data['Time'].dt.hour`, yang menambahkan kolom baru bernama hour.

2.2.3 Visualisasi Data

a. 3 Item Terlaris:

Langkah selanjutnya adalah visualisasi data untuk mendapatkan wawasan tentang item-item terlaris di kafe. Kita menggunakan bar plot untuk memvisualisasikan 3 item terlaris berdasarkan jumlah penjualan.

```
#3 Item terlaris
data_items = data.groupby('Items')['Gross Sales'].count()
data_items = pd.concat([data_items.iloc[4:], data_items.iloc[:4]])
sns.barplot(x = data['Items'].value_counts()[:3].index, y = data['Gross Sales'].value_counts()[:3].values, color = '#0E1A77')

plt.xlabel(""); plt.ylabel("")
plt.xticks(size = 13, rotation = 45)
plt.title('3 Items Paling Laris', size = 17)
plt.show()
```

Penjelasan:

- Mengelompokkan Data: Kita mengelompokkan data berdasarkan kolom Items dan menghitung jumlah penjualan bruto untuk setiap item menggunakan `data.groupby('Items')['Gross Sales'].count()`.
- Memvisualisasikan Item Terlaris: Kita membuat bar plot menggunakan `sns.barplot()` untuk menampilkan 3 item terlaris berdasarkan jumlah penjualan. Warna batang ditentukan oleh parameter `color` dan judul plot ditentukan oleh `plt.title()`.

b. Transaksi pada Bulan Juni:

Menganalisis dan memvisualisasikan jumlah transaksi pada bulan Juni untuk memahami pola penjualan selama bulan tersebut.

```
#Transaksi pada Bulan Juni
data_bulan = data.groupby('month')['Gross Sales'].count()
data_bulan = pd.concat([data_bulan.iloc[4:], data_bulan.iloc[:4]])

plt.figure(figsize=(8, 5))
sns.barplot(x = ['June'], y = data_bulan.values, color = '#0E1A77')
plt.xticks(size = 12, rotation = -30)
plt.title('Jumlah Transaksi Pada Bulan Juni', size = 16)
plt.show()
```

Penjelasan:

- Mengelompokkan Data Berdasarkan Bulan: Kita menghitung jumlah penjualan untuk setiap bulan menggunakan `data.groupby('month')['Gross Sales'].count()`.

- Memvisualisasikan Transaksi Bulan Juni: Menggunakan `sns.barplot()`, kita membuat plot batang yang menunjukkan jumlah transaksi pada bulan Juni. Ini membantu kita memahami pola penjualan selama bulan tersebut.

c. Transaksi Per Hari:

Memvisualisasikan jumlah transaksi per hari dalam seminggu untuk melihat pola penjualan berdasarkan hari dalam seminggu.

```
#Transaksi Perhari
data_perhari = data.groupby('day')['Gross Sales'].count()

plt.figure(figsize=(8, 5))
sns.barplot(x = ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'], y = data_perhari.values, color = '#0E1A77')
plt.xticks(size = 12, rotation = -30)
plt.title('Jumlah Transaksi Perhari', size = 16)
plt.show()
```

Penjelasan:

- Mengelompokkan Data Berdasarkan Hari: Kita menghitung jumlah penjualan untuk setiap hari dalam seminggu menggunakan `data.groupby('day')['Gross Sales'].count()`.
- Memvisualisasikan Transaksi Per Hari: Menggunakan `sns.barplot()`, kita membuat plot batang yang menunjukkan jumlah transaksi untuk setiap hari dalam seminggu. Ini membantu kita mengidentifikasi hari-hari dengan penjualan tertinggi dan terendah.

d. Transaksi Tiap Jam:

Memvisualisasikan jumlah transaksi per jam dalam sehari untuk melihat pola penjualan berdasarkan jam dalam sehari.

```
#Transaksi tiap jam
data_perjam = data.groupby('hour')['Gross Sales'].count()

plt.figure(figsize = (12, 6))
sns.barplot(x = data_perjam.index, y = data_perjam.values, color = '#0E1A77')
plt.xlabel('Jam', size = 15)
plt.xticks(size = 13, rotation = -30)
plt.title('Jumlah Transaksi Perjam', size = 17)
plt.show()
```

Penjelasan:

- Mengelompokkan Data Berdasarkan Jam: Kita menghitung jumlah penjualan untuk setiap jam dalam sehari menggunakan `data.groupby('hour')['Gross Sales'].count()`.
- Memvisualisasikan Transaksi Per Jam: Menggunakan `sns.barplot()`, kita membuat plot batang yang menunjukkan jumlah transaksi untuk setiap jam dalam sehari. Ini membantu kita mengidentifikasi jam-jam dengan penjualan tertinggi dan terendah.

2.2.4 Aturan Asosiasi

a. Persiapan Data

mempersiapkan data dengan membersihkan dan mengkodekan nama item. Langkah ini memastikan bahwa data siap untuk diolah menggunakan algoritma apriori.

```
## Data preparation
data['Items'] = data['Items'].apply(lambda item: item.lower())
data['Items'] = data['Items'].apply(lambda item: item.strip())
data = data[['Gross Sales', 'Items']].copy()
data.head(10)
```

	Gross Sales	Items
0	27000	kopi susu km-02 ice
1	10000	mineral
2	35000	chicken fried rice
3	28000	mushroom soup
4	25000	kopi susu ice
5	30000	latte caramel hot
6	10000	mineral
7	22500	tahu cabe garam
8	27000	chocolate ice
9	25000	moccacino ice

Penjelasan:

- Mengubah Huruf Kecil: Kita menggunakan `data['Items'].apply(lambda item: item.lower())` untuk mengubah semua nama item menjadi huruf kecil. Ini membantu menghindari inkonsistensi dalam penamaan item.
- Menghilangkan Spasi: `data['Items'].apply(lambda item: item.strip())` digunakan untuk menghilangkan spasi yang tidak diperlukan di awal dan akhir nama item.

- Menyiapkan Data: Kita memilih hanya kolom Gross Sales dan Items untuk analisis lebih lanjut dengan menggunakan `data[['Gross Sales', 'Items']].copy()`.

b. Membuat Pivot Table

Membuat pivot table untuk item dan jumlah penjualan mereka. Pivot table ini akan digunakan sebagai input untuk algoritma apriori.

```
from mlxtend.frequent_patterns import apriori, association_rules

items_count = data.groupby(['Items', 'Gross Sales'])['Items'].count().reset_index(name='Count')
print(items_count.head(10))
```

	Items	Gross Sales	Count
0	100g beef sirloin steak	77000	4
1	100g beef tenderloin steak	77000	4
2	100g beef tenderloin steak	154000	2
3	100g beef tenderloin steak	308000	1
4	150g beef sirloin steak	97500	3
5	150g beef sirloin steak	292500	1
6	150g beef tenderloin steak	97500	2
7	150g beef tenderloin steak	292500	1
8	aglio e olio	28000	5
9	aglio e olio	30000	9

```

# Pivot table
items_count_pivot = items_count.pivot_table(index='Gross Sales', columns='Items', values='Count', aggfunc='sum').fillna(0)
print('Ukuran Dataset:', items_count_pivot.shape)
print(items_count_pivot.head(10))

items_count_pivot = items_count_pivot.astype('int32')
print(items_count_pivot.head())

```

Penjelasan:

- Mengelompokkan Data: Mengelompokkan data berdasarkan kolom Items dan Gross Sales, kemudian menghitung jumlah penjualan untuk setiap kombinasi item dan penjualan. Hasilnya disimpan dalam `items_count`.
- Menampilkan Hasil: Menampilkan 10 baris pertama dari hasil pengelompokkan.
- Membuat Tabel Pivot: Menggunakan `pivot_table()` untuk membuat tabel pivot yang menampilkan jumlah penjualan untuk setiap item. Nilai kosong diisi dengan nol menggunakan `fillna(0)`.
- Mengatur Tipe Data: Mengubah tipe data tabel pivot menjadi `int32` untuk memastikan konsistensi.

- Menampilkan Hasil: Menampilkan beberapa baris pertama dari tabel pivot.

c. Encoding

```
def encode(x):  
    if x <= 0:  
        return 0  
    elif x >= 1:  
        return 1  
  
# Menerapkan fungsi encoding  
items_count_pivot = items_count_pivot.applymap(encode)  
print(items_count_pivot.head(10))
```

Penjelasan:

- Fungsi Encoding: Mendefinisikan fungsi encode() yang mengubah nilai menjadi 0 jika nilainya kurang dari atau sama dengan 0, dan mengubahnya menjadi 1 jika nilainya lebih besar dari 0.
- Menerapkan Fungsi Encoding: Menggunakan applymap() untuk menerapkan fungsi encoding pada setiap elemen di tabel pivot.
- Menampilkan Hasil: Menampilkan 10 baris pertama dari tabel pivot yang telah di-encode.

2.2.5 Penerapan Algoritma Apriori

```
#menampilkan ukuran dataset dan jumlah Gross Sales dan Items
print('Ukuran dataset:', items_count_pivot.shape)
print('Jumlah Gross Sales:', items_count_pivot.shape[0])
print('Jumlah Items:', items_count_pivot.shape[1])

Ukuran dataset: (74, 217)
Jumlah Gross Sales: 74
Jumlah Items: 217
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_
and should_run_async(code)
```

```
# Menggunakan fungsi apriori setelah konversi tipe data
support = 0.05
frequent_items = apriori(items_count_pivot, min_support=support, use_colnames=True)

# Mengurutkan dan menampilkan itemsets yang sering muncul
frequent_items_sorted = frequent_items.sort_values('support', ascending=False).head(10)
print(frequent_items_sorted)
```

	support	itemsets
12	0.229730	(custom amount)
25	0.108108	(mineral)
5	0.094595	(ayam sambal matah)
18	0.094595	(kopi susu ice)
15	0.094595	(indomie rebus km02)
29	0.081081	(regular tea ice)
26	0.067568	(mix platter)
24	0.067568	(lychee tea ice)
22	0.067568	(lemon tea ice)
21	0.067568	(lemon tea hot)

```
metric = 'lift'
min_threshold = 1.0

rules = association_rules(frequent_items, metric=metric, min_threshold=min_threshold)[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
rules.sort_values('lift', ascending=False, inplace = True)
rules.head(10)
```

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_async` will not call `transform_cell` automatically
and should_run_async(code)
```

	antecedents	consequents	support	confidence	lift
1072	(americano hot)	(indomie rebus km02, lemon tea hot, americano ...	0.054054	1.0	18.5
874	(lemon tea ice, lychee tea ice)	(indomie rebus km02, jamur crispy)	0.054054	1.0	18.5
1519	(americano ice)	(lemon tea ice, lemon tea hot, jamur crispy, L...	0.054054	1.0	18.5
891	(indomie rebus km02, lychee tea ice)	(lemon tea ice, lemon tea hot)	0.054054	1.0	18.5
890	(lemon tea hot, lychee tea ice)	(lemon tea ice, indomie rebus km02)	0.054054	1.0	18.5
1525	(custom amount, jamur crispy, kopi susu ice, L...	(lempe)	0.054054	1.0	18.5
1529	(custom amount, kopi susu ice, lychee tea ice)	(lempe, jamur crispy)	0.054054	1.0	18.5
887	(lemon tea ice, indomie rebus km02)	(lemon tea hot, lychee tea ice)	0.054054	1.0	18.5
886	(lemon tea ice, lemon tea hot)	(indomie rebus km02, lychee tea ice)	0.054054	1.0	18.5
1530	(lempe, kopi susu ice, lychee tea ice)	(custom amount, jamur crispy)	0.054054	1.0	18.5

Penjelasan:

- Menerapkan Apriori: Menggunakan fungsi `apriori()` dari `mlxtend.frequent_patterns` untuk menemukan itemsets yang sering muncul dalam data transaksi. Nilai minimum support ditentukan sebesar 0.05. Parameter `use_colnames=True` digunakan agar nama kolom asli tetap digunakan.

- Mengurutkan Itemsets: Mengurutkan itemsets berdasarkan nilai support secara menurun dan menampilkan 10 itemsets teratas.
- Menerapkan Association Rules: Menggunakan fungsi `association_rules()` untuk menemukan aturan asosiasi dari itemsets yang sering muncul. Metrik yang digunakan adalah lift dengan nilai minimum threshold sebesar 1.0.
- Menampilkan Aturan Asosiasi: Menampilkan 10 aturan asosiasi teratas berdasarkan nilai lift.

2.3 Mengintegrasikan Market Basket Analysis ke dalam Streamlit

```

%%writefile apriory.py

import streamlit as st
import numpy as np
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

# Membaca data
df = pd.read_csv('penjualancafeFix.csv')

# Mengubah format kolom 'Date' dan 'Time'
df['Date'] = pd.to_datetime(df['Date'], format='%d-%m-%Y', errors='coerce')
df['Time'] = pd.to_datetime(df['Time'], format='%H:%M:%S', errors='coerce')

# Menambahkan kolom 'month', 'day', dan 'hour'
df['month'] = df['Date'].dt.month
df['day'] = df['Date'].dt.weekday
df['hour'] = df['Time'].dt.hour

# Mengubah nama bulan dan hari
df['month'].replace([i for i in range(1, 12 + 1)], ['Januari', 'Februari', 'Maret', 'April', 'Mei', 'Juni', 'Juli', 'Agustus', 'September', 'Oktober', 'November', 'Desember'], inplace=True)
df['day'].replace([i for i in range(6+1)], ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu', 'Minggu'], inplace=True)

# Menambahkan kolom 'period_day' berdasarkan kolom 'hour'
df['period_day'] = pd.cut(df['hour'],
                          bins=[0, 11, 15, 18, 23],
                          labels=['Pagi', 'Siang', 'Sore', 'Malam'],
                          right=False)

# Menambahkan kolom 'weekday_weekend' berdasarkan kolom 'day'
df['weekday_weekend'] = df['day'].apply(lambda x: 'Weekend' if x in ['Sabtu', 'Minggu'] else 'Weekday')

st.title('Market Basket Analysis Cafe')

```

```

def get_data(period_day='', weekday_weekend='', month='', day=''):
    data = df.copy()
    filtered = data.loc[
        (data['period_day'].str.contains(period_day, case=False, na=False)) &
        (data['weekday_weekend'].str.contains(weekday_weekend, case=False, na=False)) &
        (data['month'].str.contains(month, case=False, na=False)) &
        (data['day'].str.contains(day, case=False, na=False))
    ]
    return filtered if filtered.shape[0] else 'No Result'

def user_input_features():
    item = st.selectbox('Items', df['Items'].unique())
    period_day = st.selectbox('Period Day', ['Pagi', 'Siang', 'Sore', 'Malam'])
    weekday_weekend = st.selectbox('Weekday / Weekend', ['Weekend', 'Weekday'])
    month = st.select_slider('Month', ['Januari', 'Februari', 'Maret', 'April', 'Mei', 'Juni', 'Juli', 'Agustus', 'September', 'Oktober', 'November', 'Desember'])
    day = st.select_slider('Day', ['Senin', 'Selasa', 'Rabu', 'Kamis', 'Jumat', 'Sabtu', 'Minggu'], value = 'Sabtu')

    return item, period_day, weekday_weekend, month, day

item, period_day, weekday_weekend, month, day = user_input_features()

data = get_data(period_day, weekday_weekend, month, day)

def encode(x):
    if x <= 0:
        return 0
    elif x >= 1:
        return 1

if type(data) != type('No Result'):
    items_count = data.groupby(['Items', 'Gross Sales'])['Items'].count().reset_index(name='Count')
    items_count_pivot = items_count.pivot_table(index='Gross Sales', columns='Items', values='Count', aggfunc='sum').fillna(0)
    items_count_pivot = items_count_pivot.applymap(encode)

    support = 0.05
    frequent_items = apriori(items_count_pivot, min_support=support, use_colnames=True)

```

```

metric = 'lift'
min_threshold = 1.0

rules = association_rules(frequent_items, metric=metric, min_threshold=min_threshold)[['antecedents', 'consequents', 'support', 'confidence', 'lift']]
rules.sort_values('confidence', ascending=False, inplace=True)

def parse_list(x):
    x = list(x)
    if len(x) == 1:
        return x[0]
    elif len(x) > 1:
        return ', '.join(x)
    return x

def return_item_df(items_antecedents):
    data = rules[['antecedents', 'consequents']].copy()

    data['antecedents'] = data['antecedents'].apply(parse_list)
    data['consequents'] = data['consequents'].apply(parse_list)

    result = data.loc[data['antecedents'] == items_antecedents]
    if not result.empty:
        return list(result.iloc[0, :])
    else:
        return []

if type(data) != type('No Result'):
    st.markdown('Hasil Rekomendasi:')
    rekomendasi = return_item_df(item)
    if len(rekomendasi) > 1:
        st.success(f'Dika Konsumen Membeli **{item}**, maka membeli **{rekomendasi[1]}** secara bersamaan')
    else:
        st.warning('Tidak ada rekomendasi yang tersedia untuk item yang dipilih.')
else:
    st.warning('Data tidak ditemukan untuk filter yang dipilih.')

```

Market Basket Analysis Cafe

Items

Kopi Susu KM-02 Ice



period_day

Sore



weekday / weekend

Weekday



month

Juni

Januari

Desember

day

Kamis

Senin

Minggu

Hasil Rekomendasi:

Jika Konsumen Membeli **Kopi Susu KM-02 Ice**, maka membeli **Red Velvet Latte Ice**, **Chicken Salad** secara bersamaan

BAB III PENUTUP

3.1 Kesimpulan

Analisis yang dilakukan pada dataset penjualan kafe memberikan wawasan yang signifikan tentang aktivitas penjualan dan preferensi pelanggan. Tujuan utama dari analisis ini adalah untuk memahami pola pembelian pelanggan dan memberikan informasi strategis untuk pengambilan keputusan bisnis. Dengan menggunakan teknik data mining, khususnya algoritma Apriori, ditemukan item-item yang sering dibeli bersama, yang dapat membantu dalam merancang promosi yang lebih efektif dan mengoptimalkan tata letak toko.

Dataset yang dianalisis mencakup berbagai informasi penting seperti produk yang dibeli, waktu pembelian, dan metode pembayaran, yang semuanya memberikan gambaran menyeluruh tentang operasi harian kafe. Proses analisis melibatkan beberapa langkah, termasuk pemeriksaan awal data, konversi data tanggal dan waktu, serta ekstraksi komponen waktu untuk analisis lebih lanjut. Visualisasi data dilakukan untuk menyoroti item-item terlaris, jumlah transaksi bulanan, harian, dan per jam, yang memberikan wawasan tentang tren penjualan.

Hasil dari analisis ini menunjukkan bahwa dengan memahami pola penjualan berdasarkan waktu dan produk, kafe dapat mengoptimalkan manajemen persediaan dan merancang promosi yang tepat sasaran. Selain itu, integrasi hasil analisis ke dalam aplikasi Streamlit memungkinkan visualisasi dan interaksi data yang lebih baik, membantu manajemen kafe dalam membuat keputusan yang lebih informatif dan strategis. Secara keseluruhan, analisis ini memberikan dasar yang kuat bagi kafe untuk meningkatkan kinerja penjualan dan kepuasan pelanggan.